

Руководство по запуску и настройке

Finance Helper

Релиз-версия программного продукта для учёта личных и совместных финансов с использованием Telegram-бота, Mini App и микросервисной архитектуры

Параметр	Значение
Назначение	Запуск, первичная настройка, проверка работоспособности и подготовка системы к демонстрации.
Актуальная схема запуска	Облачный сервер без Docker Compose. Сервисы запускаются как отдельные Python-процессы.
Рабочая директория	finance_helper/source_files
Mini App	Работает через постоянный HTTPS-домен. Ngrok не используется.
Состав поставки	Telegram-бот, API Gateway, Finance Service, Analytics Service, PostgreSQL, Mini App, тесты, сценарии и руководство.

Содержание

1. Назначение и основные возможности
2. Состав системы
3. Структура архива и рабочая директория
4. Системные требования
5. Переменные окружения
6. Подготовка облачного сервера и установка зависимостей
7. Настройка базы данных и миграции
8. Запуск и остановка сервисов
9. Настройка домена и Mini App
10. Запуск demo seed
11. Тестирование
12. Первичная проверка работоспособности
13. Безопасность и подготовка к публикации
14. Документация

1. Назначение и основные возможности

Finance Helper - это программный продукт для ведения личного и совместного бюджета в Telegram. Система объединяет Telegram-бота, Mini App и микросервисную серверную часть. Актуальная релизная версия рассчитана на запуск на облачном сервере без использования Docker Compose и без временного туннеля ngrok.

- учёт доходов и расходов;
- добавление операций текстом, командами и кнопками меню;
- указание даты операции, в том числе задним числом;
- работа с категориями, лимитами и рабочими пространствами;
- просмотр операций, отчётов и аналитики;
- экспорт данных в CSV и XLSX;
- импорт банковских выписок;
- использование Mini App для расширенного просмотра данных;
- работа через постоянный HTTPS-домен для Mini App.

2. Состав системы

Компонент	Назначение
bot-service	Telegram-бот и основной пользовательский интерфейс. Запускается как отдельный Python-процесс.
api-gateway	Единая точка входа для запросов, маршрутизация между сервисами и выдача Mini App.
finance-service	Хранение и обработка пользователей, операций, категорий, лимитов, рабочих пространств и импорта выписок.
analytics-service	Отчёты, аналитика, экспорт CSV/XLSX и данные для Mini App.
PostgreSQL	Основная база данных проекта. Может быть установлена на том же сервере или вынесена отдельно.
Mini App	Веб-интерфейс для просмотра дашборда, динамики, категорий, лимитов и участников пространства.
Nginx / reverse proxy	Проксирование HTTPS-запросов с домена на api-gateway. Не является частью бизнес-логики проекта, но требуется для серверного доступа по домену.

Сервисы остаются микросервисными по архитектуре, но в актуальной схеме они запускаются напрямую на сервере, а не в контейнерах Docker.

3. Структура архива и рабочая директория

Ниже приведена рекомендуемая структура релизного архива. Основная работа по запуску, тестированию и настройке выполняется из каталога `finance_helper/source_files`.

```

Finance_Helper/
├── README.md
├── finance_helper/
│   ├── run_and_configuration_guide/
│   │   └── Finance_Helper_Guide.docx / Finance_Helper_Guide.pdf
│   └── source_files/
│       ├── .env.example
│       ├── .gitignore
│       ├── Makefile
│       ├── pytest.ini
│       ├── requirements.txt
│       ├── docs/
│       ├── scripts/
│       ├── services/
│       │   ├── bot-service/
│       │   ├── api-gateway/
│       │   ├── finance-service/
│       │   └── analytics-service/
│       └── tests/

```

В релизный архив не должны входить настоящий файл `.env`, папка `.git`, локальные виртуальные окружения, кэш Python и служебные файлы IDE.

4. Системные требования

Компонент	Минимальное требование
ОС сервера	Linux-сервер, рекомендуется Ubuntu 22.04 LTS или новее.
Python	Python 3.11 или новее, модуль <code>venv</code> и <code>pip</code> .
PostgreSQL	PostgreSQL 14 или новее.
Nginx	Рекомендуется для проксирования домена и HTTPS-запросов к <code>api-gateway</code> .
Systemd	Рекомендуется для постоянного запуска сервисов после перезагрузки сервера.
Telegram	Аккаунт Telegram и токен бота, полученный через BotFather.
Интернет и домен	Требуются для доступа к Telegram Bot API и открытия Mini App по HTTPS.
OCR-зависимости	Для сценариев с чеками рекомендуется установить <code>tesseract-ocr</code> , <code>tesseract-ocr-rus</code> и <code>tesseract-ocr-eng</code> .

Пример установки системных пакетов на Ubuntu:

```

sudo apt update
sudo apt install -y python3 python3-venv python3-pip postgresql postgresql-contrib nginx
sudo apt install -y tesseract-ocr tesseract-ocr-rus tesseract-ocr-eng

```

5. Переменные окружения

Перед запуском необходимо создать файл `.env` на основе шаблона `.env.example` и заполнить обязательные параметры. Файл `.env` содержит секреты и не должен публиковаться в GitHub, передаваться в открытом архиве или включаться в отчёт.

Переменная	Назначение
------------	------------

BOT_TOKEN	Токен Telegram-бота, полученный у BotFather.
INTERNAL_API_KEY	Внутренний ключ взаимодействия сервисов; должен быть одинаковым для сервисов, которые проверяют внутренние запросы.
MINIAPP_SIGNING_SECRET	Секретный ключ для подписи токенов Mini App.
MINIAPP_PUBLIC_URL	Публичный HTTPS-адрес Mini App на домене, например <code>https://your-domain.example/miniapp/app</code> .
POSTGRES_HOST / POSTGRES_PORT	Адрес и порт PostgreSQL. При запуске на одном сервере обычно используются 127.0.0.1 и 5432.
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Имя базы данных, пользователь и пароль PostgreSQL.
FINANCE_URL	Внутренний URL finance-service, обычно <code>http://127.0.0.1:8001</code> .
ANALYTICS_URL	Внутренний URL analytics-service, обычно <code>http://127.0.0.1:8002</code> .
GATEWAY_URL	Внутренний URL api-gateway для bot-service, обычно <code>http://127.0.0.1:8000</code> .
UPLOAD_DIR	Каталог для загружаемых файлов, например <code>/data/finance_helper_uploads</code> .

Создание файла окружения:

```
cd finance_helper/source_files
cp .env.example .env
nano .env
```

Пример базовых значений для запуска всех сервисов на одном сервере:

```
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
FINANCE_URL=http://127.0.0.1:8001
ANALYTICS_URL=http://127.0.0.1:8002
GATEWAY_URL=http://127.0.0.1:8000
MINIAPP_PUBLIC_URL=https://your-domain.example/miniapp/app
```

6. Подготовка облачного сервера и установка зависимостей

Рекомендуемый путь размещения проекта на сервере может быть любым. В примерах ниже используется каталог `/opt/finance_helper`. Если проект находится в другой директории, путь в командах необходимо заменить на актуальный.

6.1. Создание виртуального окружения

```
cd /opt/finance_helper/finance_helper/source_files
python3 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

6.2. Проверка установки зависимостей

```
python -m compileall services tests scripts
```

Если команда завершается без ошибок, основные Python-модули проекта доступны и синтаксически корректны.

7. Настройка базы данных и миграции

Для работы `finance-service` требуется PostgreSQL. База данных может быть создана вручную через `psql` или через административную панель облачного провайдера.

7.1. Создание пользователя и базы данных PostgreSQL

```
sudo -u postgres psql
CREATE USER finance_user WITH PASSWORD 'strong_password_here';
CREATE DATABASE finance_db OWNER finance_user;
\q
```

После создания базы данных необходимо указать те же значения в `.env`: `POSTGRES_DB`, `POSTGRES_USER` и `POSTGRES_PASSWORD`.

7.2. Применение миграций

```
cd /opt/finance_helper/finance_helper/source_files/services/finance-service
../../venv/bin/python -m alembic upgrade head
```

Миграции создают структуру базы данных, необходимую для хранения пользователей, операций, категорий, лимитов, рабочих пространств и данных импорта.

8. Запуск и остановка сервисов

В актуальной версии проекта Docker Compose не используется. Каждый сервис запускается как отдельный Python-процесс. Для ручной проверки можно открыть четыре отдельные консоли или использовать команды Makefile.

8.1. Быстрый ручной запуск через Makefile

```
cd /opt/finance_helper/finance_helper/source_files

make run-finance
make run-analytics
make run-gateway
make run-bot
```

Команды запускают соответственно `finance-service`, `analytics-service`, `api-gateway` и `bot-service`. Перед запуском должен быть заполнен файл `.env` и применены миграции базы данных.

8.2. Ручной запуск без Makefile

```
# finance-service
cd /opt/finance_helper/finance_helper/source_files/services/finance-service
../../venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8001

# analytics-service
cd /opt/finance_helper/finance_helper/source_files/services/analytics-service
```

```

../../venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8002

# api-gateway
cd /opt/finance_helper/finance_helper/source_files/services/api-gateway
../../venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 8000

# bot-service
cd /opt/finance_helper/finance_helper/source_files/services/bot-service
../../venv/bin/python -m app.main

```

8.3. Рекомендуемый запуск через systemd

Для постоянной работы на сервере рекомендуется создать отдельные systemd-службы. Ниже приведён пример для api-gateway; по аналогии создаются службы для finance-service, analytics-service и bot-service.

```

# /etc/systemd/system/finance-helper-gateway.service

[Unit]
Description=Finance Helper API Gateway
After=network.target postgresql.service

[Service]
User=www-data
WorkingDirectory=/opt/finance_helper/finance_helper/source_files/services/api-gateway
EnvironmentFile=/opt/finance_helper/finance_helper/source_files/.env
ExecStart=/opt/finance_helper/finance_helper/source_files/venv/bin/python -m uvicorn
app.main:app --host 127.0.0.1 --port 8000
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

После создания service-файлов необходимо выполнить:

```

sudo systemctl daemon-reload
sudo systemctl enable finance-helper-finance
sudo systemctl enable finance-helper-analytics
sudo systemctl enable finance-helper-gateway
sudo systemctl enable finance-helper-bot

sudo systemctl start finance-helper-finance
sudo systemctl start finance-helper-analytics
sudo systemctl start finance-helper-gateway
sudo systemctl start finance-helper-bot

```

Проверка статуса службы:

```

sudo systemctl status finance-helper-gateway
sudo journalctl -u finance-helper-gateway -f

```

9. Настройка домена и Mini App

Mini App открывается по адресу, заданному в переменной `MINIAPP_PUBLIC_URL`. В актуальной версии используется постоянный HTTPS-домен. Ngrok не требуется и не используется.

Пример значения:

```
MINIAPP_PUBLIC_URL=https://your-domain.example/miniapp/app
```

Домен должен проксировать запросы к `api-gateway`. Для Mini App особенно важны пути `/miniapp/app` и `/miniapp/public/...`. Также через `api-gateway` проходят API-запросы Mini App.

9.1. Пример настройки Nginx

```
server {
    server_name your-domain.example;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

После настройки домена и HTTPS необходимо проверить, что адрес `/miniapp/app` открывается в браузере и внутри Telegram. Если домен меняется, значение `MINIAPP_PUBLIC_URL` нужно обновить в `.env` и перезапустить `bot-service`.

10. Запуск demo seed

Для заполнения системы демонстрационными данными используется сценарий `scripts/seed_demo.py`. Перед запуском должен работать `api-gateway`, а в `.env` должны быть заполнены `DEMO_TELEGRAM_ID` и `DEMO_TELEGRAM_USERNAME`.

```
cd /opt/finance_helper/finance_helper/source_files
source venv/bin/activate
python scripts/seed_demo.py
```

11. Тестирование

Автоматизированные тесты запускаются из рабочей директории проекта. Часть smoke-тестов требует, чтобы `api-gateway` был запущен.

```
cd /opt/finance_helper/finance_helper/source_files
source venv/bin/activate
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python -m pytest -q
```

Для подробной диагностики можно использовать расширенный вывод `pytest`:

```
PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python -m pytest -rs -vv
```

Если `api-gateway` доступен по домену, для smoke-тестов можно явно указать адрес проверки:

```
TEST_GATEWAY_URL=https://your-domain.example PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 python -m  
pytest tests/test_gateway_health.py tests/test_security.py
```

12. Первичная проверка работоспособности

После запуска сервисов необходимо выполнить ручную проверку основных сценариев.

Что проверить	Ожидаемый результат
GET /health на api-gateway	Ответ 200 ОК и тело {"status":"ok"}.
GET /health на finance-service	Финансовый сервис доступен и отвечает без ошибки.
GET /health на analytics-service	Сервис аналитики доступен и отвечает без ошибки.
Команда /start в Telegram	Бот отвечает приветствием и отображает клавиатуру.
Добавление операции	Операция создаётся, сохраняется в PostgreSQL и отображается в списке последних операций.
Редактирование и удаление операции	Изменения корректно сохраняются, удалённая операция больше не отображается пользователю.
Статистика за месяц	Формируется сводка с доходами, расходами и балансом.
Mini App	Открывается по публичному HTTPS-адресу и показывает данные текущего пользователя.
Экспорт CSV/XLSX	Файл успешно формируется и отправляется пользователю.
Импорт выписки	Файл принимается, предварительный просмотр формируется, импорт завершается корректно.

Быстрая проверка health-check через curl:

```
curl http://127.0.0.1:8000/health  
curl http://127.0.0.1:8001/health  
curl http://127.0.0.1:8002/health
```

13. Безопасность и подготовка к публикации

Перед публикацией проекта преподавателю необходимо убедиться, что архив не содержит секретов и локальных служебных файлов.

Что проверить	Что должно быть в релизной версии
Файл .env	Не должен входить в архив и репозиторий. Вместо него должен быть только .env.example.
BOT_TOKEN	Не должен быть опубликован. При утечке токен нужно перевыпустить через BotFather.
INTERNAL_API_KEY и MINIAPP_SIGNING_SECRET	Не должны быть опубликованы. При утечке значения нужно заменить.
Папка .git	Не должна входить в архив для проверки, если преподавателю нужен только исходный код релизной версии.
venv, __pycache__, .pytest_cache	Не должны входить в релизный архив.
Docker-файлы	В актуальной версии без Docker не требуются.
README и руководство	Должны описывать серверный запуск, домен и отсутствие ngrok.

Рекомендуемый минимальный .gitignore:

```
.env
.env.*
!.env.example
venv/
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.coverage
htmlcov/
.vscode/
.idea/
*.log
```

14. Документация

Файл	Назначение
README.md	Краткое описание проекта, состава поставки, возможностей и базового запуска.
Finance_Helper_Guide.docx / Finance_Helper_Guide.pdf	Подробное руководство по запуску и настройке на облачном сервере.
docs/test_scenarios.md	Сценарии ручного тестирования ключевых функций.
.env.example	Безопасный шаблон переменных окружения без реальных секретов.
requirements.txt	Общий список Python-зависимостей для серверного запуска.

Если способ развёртывания изменяется, в первую очередь необходимо обновить README.md, .env.example, руководство по запуску, сценарии тестирования и инструкции для Mini App.

Локальная демонстрация — дополнение от 09.09.2026

Это дополнение описывает запуск на localhost. Остальные страницы руководства относятся к серверному развёртыванию. Для локального Mini App домен, HTTPS и Telegram-бот не требуются.

Требуются Python 3.11 и PostgreSQL 14+. Из корня репозитория перейдите в `finance_helper/source_files`. Создайте venv: `python -m venv venv`. Активируйте его: `source venv/bin/activate` (Linux/macOS) или `.\venv\Scripts\Activate.ps1` (PowerShell).

Установите зависимости: `python -m pip install -r requirements.txt`. Скопируйте `.env.example` в `.env`. Создайте отдельную локальную роль и базу PostgreSQL и заполните `POSTGRES_HOST`, `POSTGRES_PORT`, `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`.

Из `source_files` выполните: `python scripts/run_demo.py`. Команда применяет миграции, запускает три сервиса на `127.0.0.1:8100-8102` и печатает ссылку Mini App. Откройте её в браузере. Ссылка содержит токен и не должна публиковаться.

Остановка: `Ctrl+C` в терминале `launcher`. Данные остаются в выбранной БД. Если порты заняты: `python scripts/run_demo.py --port 8200`. Проверка с автоматической остановкой: `python scripts/run_demo.py --smoke-test`.

Пошаговая инструкция с SQL создания БД и командами тестирования для PowerShell: `docs/local-run.md` в корне репозитория. Launcher не устанавливает зависимости и не создаёт роль/БД PostgreSQL.